方法

GRPO

GRPO[9] 是我们用于训练 DeepSeek-R1-Zero 和 DeepSeek-R1 的 RL 算法。该算法最初被提出，旨在简化近端策略优化（PPO）的训练过程并降低其资源消耗[31]，，PPO 广泛用于 LLM 的 RL 阶段[32]。GRPO 的流程如扩展数据图 2 所示。

对于每个问题 q，GRPO 从旧策略中采样一组输出 $\{o_1, o_2, \ldots, o_G\}$，旧策略为 $\pi_{\theta_{\text{old}}}$，然后通过最大化以下目标来优化策略模型 π_θ：

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{\text{old}}}(O|q)]$$
$$\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{\text{old}}}(o_i|q)}A_i,\ \text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{\text{old}}}(o_i|q)}, 1-\epsilon, 1+\epsilon\right)A_i\right)\right. \tag{1}$$
$$\left. - \beta \mathbb{D}_{KL}(\pi_\theta||\pi_{\text{ref}})\right),$$

$$\mathbb{D}_{\text{KL}}(\pi_\theta||\pi_{\text{ref}}) = \frac{\pi_{\text{ref}}(o_i|q)}{\pi_\theta(o_i|q)} - \log\frac{\pi_{\text{ref}}(o_i|q)}{\pi_\theta(o_i|q)} - 1, \tag{2}$$

其中，π_{ref} 为参考策略，ϵ 和 β 为超参数，A_i 为优势值，使用与每组输出对应的一组奖励 $\{r_1, r_2, \ldots, r_G\}$ 计算得到：

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \cdots, r_G\})}{\text{std}(\{r_1, r_2, \cdots, r_G\})}. \tag{3}$$

我们在补充信息第 1.3 节中给出了 GRPO 与 PPO 的比较。

奖励设计

奖励是训练信号的来源，决定了 RL 优化的方向。对于 DeepSeek-R1-Zero，我们使用基于规则的奖励，为数学、编程和逻辑推理领域的数据提供精确反馈。对于 DeepSeek-R1，我们扩展了这一方法，将面向推理的数据所用的基于规则的奖励与通用数据所用的基于模型的奖励相结合，从而增强学习过程在不同领域中的适应性。

基于规则的奖励。我们基于规则的奖励系统主要包含两类奖励：准确性奖励和格式奖励。

准确性奖励评估回答是否正确。例如，对于结果确定的数学问题，要求模型以指定格式（例如，置于方框内）给出最终答案，以便基于规则可靠地验证其正确性。同样，对于编程竞赛提示，可以使用编译器，根据一组预定义的测试用例评估模型的回答，从而生成关于正确性的客观反馈。

格式奖励通过强制执行特定的格式要求，对准确性奖励模型起到补充作用。具体而言，模型受到激励，将其推理过程封装在指定标签内，即 <think> and </think>. 这确保了模型的思考过程得到明确标示，从而增强可解释性，并便于后续分析。

$$\text{Reward}_{\text{rule}} = \text{Reward}_{\text{acc}} + \text{Reward}_{\text{format}} \tag{4}$$

准确性奖励和格式奖励以相同的权重组合。值得注意的是，我们不对推理任务应用神经奖励模型，无论是基于结果的还是基于过程的。这一决定基于我们的观察：神经奖励模型在大规模 RL 过程中容易受到奖励破解的影响。此外，重新训练此类模型需要大量计算资源，并会增加训练流程的复杂性，从而使整个优化过程更加复杂。

基于模型的奖励。对于通用数据，我们采用奖励模型来捕捉复杂且细微场景中的人类偏好。我们基于 DeepSeek-V3 流程，使用分布相似的偏好对和训练提示。对于有用性，我们仅关注最终总结，确保评估着重考察回答对用户的实用性和相关性，同时尽量减少对底层推理过程的干扰。对于无害性，我们评估模型的整个回答，包括推理过程和总结，以识别并减轻生成过程中可能出现的任何潜在风险、偏见或有害内容。

有用性奖励模型。在训练有用性奖励模型时，我们首先使用补充信息第 2.2 节中列出的 Arena-Hard 提示格式向 DeepSeek-V3 提供提示，生成偏好对，每对包含一个用户查询和两个候选回答。对于每个偏好对，我们向 DeepSeek-V3 查询四次，将回答随机分配为 Response A 或 Response B，以减轻位置偏差。最终偏好分数通过对四次独立判断取平均值来确定，仅保留分数差（Δ）超过 1 的偏好对，以确保差异具有意义。此外，为尽量减少与长度相关的偏差，我们确保整个数据集中被选中的回答和被拒绝的回答长度相近。我们总共整理了 66,000 个数据对用于训练奖励模型。该数据集中使用的提示均为非推理问题，来源于公开可用的开源数据集，或明确同意共享其数据以改进模型的用户。我们的奖励模型架构与 DeepSeek-R1 一致，并额外增加了一个用于预测标量偏好分数的奖励头。

$$\text{Reward}_{\text{helpful}} = \text{RM}_{\text{helpful}}(\text{Response}_A, \text{Response}_B) \tag{5}$$

有用性奖励模型以 256 的批量大小、6×10^{-6} 的学习率在训练数据集上训练一个轮次。训练期间的最大序列长度设为 8,192 个 token，而奖励模型推理期间不设明确的长度限制。

安全性奖励模型。为了评估和提高模型的安全性，我们整理了一个包含 106,000 条提示的数据集，并根据预先定义的安全准则，将模型生成的回答标注为‘安全’或‘不安全’。与有用性奖励模型所使用的成对损失不同，安全性奖励模型采用逐点方法进行训练，以区分安全与不安全的回答。训练超参数与有用性奖励模型相同。

$$\text{Reward}_{\text{safety}} = \text{RM}_{\text{safety}}(\text{Response}) \tag{6}$$

对于一般查询，每个实例被归类为属于安全性数据集或有用性数据集。分配给每个查询的通用奖励 $\text{Reward}_{\text{general}}$ 对应于其所属数据集中定义的相应奖励。

训练细节

DeepSeek-R1-Zero 的训练细节。为训练 DeepSeek-R1-Zero，我们将学习率设为 3×10^{-6}，将 Kullback–Leibler（KL）系数设为 0.001，并将 rollout 的采样温度设为 1。对于每个问题，我们采样 16 个输出，在第 8.2k 步之前，最大长度为 32,768 个 token，此后为 65,536 个 token。因此，DeepSeek-R1-Zero 的性能和回答长度均在第 8.2k 步出现显著跃升，训练持续进行至总计 10,400 步，对应 1.6 个训练轮次。每个训练步骤包含 32 个不同的问题，因此每步的训练批量大小为 512。每隔 400 步，我们将参考模型替换为

方法

GRPO

GRPO[9] is the RL algorithm we use to train DeepSeek-R1-Zero and DeepSeek-R1. The algorithm was originally proposed to simplify the training process of proximal policy optimization (PPO) and reduce its resource consumption[31], ; PPO is widely used in the RL stage of LLMs[32]. The GRPO procedure is shown in Extended Data Fig. 2.

For each question q, GRPO samples a group of outputs $\{o_1, o_2, \ldots, o_G\}$ from the old policy $\pi_{\theta_{\text{old}}}$, and then optimizes the policy model π_θ by maximizing the following objective:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{\text{old}}}(O|q)]$$
$$\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{\text{old}}}(o_i|q)}A_i,\ \text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{\text{old}}}(o_i|q)}, 1-\epsilon, 1+\epsilon\right)A_i\right)\right. \tag{1}$$
$$\left. - \beta \mathbb{D}_{KL}(\pi_\theta||\pi_{\text{ref}})\right),$$

$$\mathbb{D}_{\text{KL}}(\pi_\theta||\pi_{\text{ref}}) = \frac{\pi_{\text{ref}}(o_i|q)}{\pi_\theta(o_i|q)} - \log\frac{\pi_{\text{ref}}(o_i|q)}{\pi_\theta(o_i|q)} - 1, \tag{2}$$

Here, π_{ref} is the reference policy, ϵ and β are hyperparameters, and A_i is the advantage, calculated using a set of rewards $\{r_1, r_2, \ldots, r_G\}$ corresponding to each group of outputs:

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \cdots, r_G\})}{\text{std}(\{r_1, r_2, \cdots, r_G\})}. \tag{3}$$

We provide a comparison of GRPO and PPO in Section 1.3 of the Supplementary Information.

Reward Design

Rewards are the source of the training signal and determine the direction of RL optimization. For DeepSeek-R1-Zero, we use rule-based rewards to provide precise feedback for data in mathematics, programming, and logical reasoning. For DeepSeek-R1, we extend this approach by combining rule-based rewards for reasoning-oriented data with model-based rewards for general-purpose data, thereby enhancing the adaptability of the learning process across different domains.

Rule-based rewards. Our rule-based reward system primarily comprises two types of rewards: accuracy rewards and format rewards.

Accuracy rewards assess whether responses are correct. For example, for mathematics problems with determinate results, the model is required to provide the final answer in a specified format (e.g., enclosed in a box), enabling reliable rule-based verification of its correctness. Similarly, for competitive programming prompts, a compiler can be used to evaluate the model's responses against a set of predefined test cases, thereby generating objective feedback on correctness.

The format reward complements the accuracy reward model by enforcing specific formatting requirements. Specifically, the model is incentivized to enclose its reasoning process within designated tags, namely <think> and </think>.. This ensures that the model's thinking process is clearly marked, thereby enhancing interpretability and facilitating subsequent analysis.

$$\text{Reward}_{\text{rule}} = \text{Reward}_{\text{acc}} + \text{Reward}_{\text{format}} \tag{4}$$

The accuracy reward and format reward are combined with equal weights. Notably, we do not apply neural reward models to reasoning tasks, whether outcome-based or process-based. This decision is based on our observation that neural reward models are susceptible to reward hacking during large-scale RL.

Furthermore, retraining such models requires substantial computational resources and increases the complexity of the training pipeline, thereby making the overall optimization process more complex.

Model-based rewards. For general data, we use a reward model to capture human preferences in complex and nuanced scenarios. Following the DeepSeek-V3 pipeline, we use preference pairs and training prompts with similar distributions. For helpfulness, we focus solely on the final summary, ensuring that the evaluation emphasizes the response's usefulness and relevance to the user while minimizing interference with the underlying reasoning process. For harmlessness, we evaluate the model's entire response, including the reasoning process and summary, to identify and mitigate any potential risks, biases, or harmful content that may arise during generation.

Helpfulness reward model. When training the helpfulness reward model, we first prompt DeepSeek-V3 using the Arena-Hard prompt format listed in Section 2.2 of the Supplementary Information to generate preference pairs, each containing a user query and two candidate responses. For each preference pair, we query DeepSeek-V3 four times, randomly assigning the responses as Response A or Response B to mitigate position bias. The final preference scores are determined by averaging four independent judgments, and only preference pairs with a score difference (Δ) greater than 1 are retained to ensure meaningful differences. In addition, to minimize length-related bias, we ensure that the chosen and rejected responses are of similar lengths across the entire dataset. In total, we compiled 66,000 data pairs to train the reward model. The prompts used in this dataset are all non-reasoning questions, sourced from publicly available open-source datasets or from users who explicitly consented to sharing their data to improve the model. Our reward model architecture is consistent with DeepSeek-R1, with an additional reward head for predicting scalar preference scores.

$$\text{Reward} = \text{RM}(\text{Response}, \text{Response})_{\text{helpful helpful A B}} \tag{5}$$

The helpfulness reward model is trained for one epoch on the training dataset with a batch size of 256 and a learning rate of 6×10^{-6}. The maximum sequence length during training is set to 8,192 tokens, while no explicit length limit is imposed during reward model inference.

Safety reward model. To evaluate and improve the model's safety, we curated a dataset containing 106,000 prompts and labeled the model-generated responses as ‘safe’ or ‘unsafe’ according to predefined safety guidelines. Unlike the pairwise loss used by the helpfulness reward model, the safety reward model is trained using a pointwise approach to distinguish safe responses from unsafe ones. The training hyperparameters are the same as those of the helpfulness reward model.

$$\text{Reward}_{\text{safety}} = \text{RM}_{\text{safety}}(\text{Response}) \tag{6}$$

For general queries, each instance is classified as belonging to either the safety dataset or the helpfulness dataset. The general reward $\text{Reward}_{\text{general}}$ assigned to each query corresponds to the respective reward defined in the dataset to which it belongs.

Training Details

Training details for DeepSeek-R1-Zero. To train DeepSeek-R1-Zero, we set the learning rate to 3×10^{-6}, the Kullback–Leibler (KL) coefficient to 0.001, and the rollout sampling temperature to 1. For each question, we sample 16 outputs, with a maximum length of 32,768 tokens before step 8.2k and 65,536 tokens thereafter. Consequently, both the performance and response length of DeepSeek-R1-Zero show a significant jump at step 8.2k. Training continues for a total of 10,400 steps, corresponding to 1.6 training epochs. Each training step includes 32 distinct questions, so the training batch size per step is 512. Every 400 steps, we replace the reference model with